

PROGRAMAÇÃO FUNCIONAL

FUNÇÕES DE ORDEM SUPERIOR

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLOGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

OBJETIVOS DA AULA

- **Identificar** a repetição de **padrões estruturais** em funções **recursivas**.
- Compreender o conceito de **funções** como cidadãs de **primeira classe**.
- Desenvolver a capacidade de **generalizar** comportamentos **injetando funções** como **parâmetros**.

MOTIVAÇÃO

1. $\text{doubleList} :: [\text{Int}] \rightarrow [\text{Int}]$
2. $\text{doubleList} [] = []$
3. $\text{doubleList} (x : xs) = 2 * x : \text{doubleList} xs$

1. $\text{notList} :: [\text{Bool}] \rightarrow [\text{Bool}]$
2. $\text{notList} [] = []$
3. $\text{notList} (x : xs) = \text{not } x : \text{notList } xs$

ANÁLISE CRÍTICA DAS ESTRUTURAS

O que permanece idêntico? A **travessia da lista** (o caso base `[]` e a desestruturação `(x:xs)`) e a **reconstrução** da lista através do operador `cons (:`).

ANÁLISE CRÍTICA DAS ESTRUTURAS

O que se altera? Exclusivamente a **operação** matemática **aplicada** à **cabeça** da lista (a variável x). Na primeira função, avalia-se $(x + 1)$; na segunda, $(x * 2)$.

ANÁLISE CRÍTICA DAS ESTRUTURAS

Problema: Se for necessário **subtrair, dividir** ou **aplicar um logaritmo**, teríamos que escrever uma **nova função** recursiva **estruturalmente idêntica**.

GENERALIZAÇÃO DO COMPORTAMENTO

Para **abstrair** a repetição, **extraímos** a **parte que muda** (a operação matemática) e a transformamos em um **parâmetro**.

Em **linguagens funcionais**, isso significa passar uma **função** como **argumento**:

GENERALIZAÇÃO DO COMPORTAMENTO

- 1.-- A funcao 'f' encapsula a **operacao**
2. $\text{aplicaOperacao} :: (\text{Int} \rightarrow \text{Int}) \rightarrow [\text{Int}] \rightarrow [\text{Int}]$
3. $\text{aplicaOperacao} _ [] = []$
4. $\text{aplicaOperacao } f (x:xs) = f\ x : \text{aplicaOperacao } f\ xs$

GENERALIZAÇÃO DO COMPORTAMENTO

Agora, podemos recriar as funções anteriores aproveitando a abstração:

1. `incrementa x = x + 1`

2. `somaUmRefatorada xs = aplicaOperacao
 incrementa xs`

FUNÇÕES DE ORDEM SUPERIOR

Definição Formal: Uma **função** é classificada como de "**Ordem Superior**" se satisfaz pelo menos **uma** das seguintes propriedades:

1. Recebe **uma** ou **mais funções** como **argumentos** de entrada.
2. **Retorna** uma **função** como seu valor de **saída**.

MAP

A **generalização** que criamos no slide anterior (**aplicaOperacao**) é, de fato, a **função nativa map** da biblioteca **padrão** (Prelude) do **Haskell**, cuja **assinatura** polimórfica é:

$$\text{map} :: (a \rightarrow b) \rightarrow [a] \rightarrow [b]$$

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

Criar **funções auxiliares** nomeadas (como incrementa) apenas para passá-las como **parâmetro** pode **poluir** o escopo global.

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

Solução: O **Cálculo Lambda** fundamenta a **criação de funções anônimas** construídas em tempo de **execução** (on-the-fly).

Sintaxe em Haskell: `\argumento -> expressao`

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

- 1.-- Utilizando map com funcoes lambda
- 2.listaIncrementada = map (\x -> x + 1) [1, 2, 3]
- 3.listaDobrada = map (\x -> x * 2) [1, 2, 3]

exemploSomaDois = map

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

Em cenários **reais** de engenharia de software, frequentemente lidamos com **coleções de dados** que possuem **dependência posicional** (dados pareados).

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

Problema de Negócio:

Considere um sistema de gestão de estoque. Possuímos **duas** listas **distintas**: uma contendo o **preço** unitário de diversos **produtos** e outra contendo a **quantidade** adquirida de cada um desses **produtos**.

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

Exemplo de Dados:

- `precos = [10.50, 20.00, 5.25]`
- `quantidades = [2, 3, 10]`
- Resultado Esperado: `[21.00, 60.00, 52.50]`

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

Para processar as **duas listas** simultaneamente, a **biblioteca** padrão do Haskell nos fornece a função de ordem superior **zipWith**.

- A assinatura de **zipWith** exige uma função que receba **dois argumentos**: $(a \rightarrow b \rightarrow c) \rightarrow [a] \rightarrow [b] \rightarrow [c]$.

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

A Solução Ingênua:

Definir uma função nomeada exclusivamente para realizar essa multiplicação.

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

- 1. Cria-se uma função auxiliar no escopo global
calculaSubTotal :: Double -> Double -> Double
calculaSubTotal preco qtd = preco * qtd
- 2. Aplica-se a função via zipWith
subTotais = zipWith calculaSubTotal precos
quantidades

FUNÇÕES ANÔNIMAS (EXPRESSÕES LAMBDA)

A Solução Elegante: Lambda com Múltiplos Argumentos

```
precos      = [10.50, 20.00, 5.25]  
quantidades = [2.0, 3.0, 10.0]
```

```
-- Resolução direta e concisa:  
subTotais = zipWith (\preco qtd -> preco * qtd)  
precos quantidades
```


EXERCÍCIOS

-- Exercício 1: Inversão de Lógica Condicional (Filter)

-- Deve receber um predicado (função que retorna Bool) e uma lista retornando apenas os elementos que satisfazem a condição.

`meuFilter :: (a -> Bool) -> [a] -> [a]`
`meuFilter = undefined`

EXERCÍCIOS

- Exercício 2: Aplicação Sucessiva
- Objetivo: Receber uma função 'f' e um valor 'x', e aplicar a função duas vezes sucessivas sobre o valor. Exemplo: $f(f\ x)$.

`aplicaDuasVezes :: (a -> a) -> a -> a`
`aplicaDuasVezes = undefined`

EXERCÍCIOS

-- Exercício 3: Composição de Operações de Lista

-- Objetivo: Utilizando as funções nativas 'map' e 'filter' (com lambdas), descartar os números ímpares de uma lista e elevar os pares remanescentes ao quadrado.

```
somaQuadradosPares :: [Int] -> [Int]  
somaQuadradosPares = undefined
```


EXERCÍCIOS

- Exercício 4: Paradigma ZipWith
- Objetivo: Receber uma operação binária arbitrária e aplicá-la elemento a elemento entre duas listas, até que a mais curta termine.

$\text{aplicaBinaria} :: (a \rightarrow b \rightarrow c) \rightarrow [a] \rightarrow [b] \rightarrow [c]$
 $\text{aplicaBinaria} = \text{undefined}$

EXERCÍCIOS

- Exercício 5: Mapeamento de Predicados (Any)
- Objetivo: Retornar True se ao menos um elemento da lista satisfazer o predicado. A avaliação deve ser interrompida no primeiro sucesso.

$\text{algumSatisfaz} :: (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow \text{Bool}$
 $\text{algumSatisfaz} = \text{undefined}$

DÚVIDAS???